

CHAPTER: 5

★ "TIME AND SPACE COMPLEXITY" ★

TIME AND SPACE COMPLEXITY:-

Definition:-

"The measure of time taken by an algorithm is called 'TIME COMPLEXITY' and the measure of space (memory) consumed by an algorithm is called 'SPACE COMPLEXITY'."

Explanation:-

Time and space complexity are basically a tool to show how an algorithm's speed and memory usage changes depending on the provided input. It relates how ^{much} time an algorithm would take from start to end in relation with input.

WHY DO WE NEED IT:-

Our purpose in data structures is to efficiently utilize memory while also making

Date: ___/___/___

Day: ___

our algorithms as efficient as we can. An algorithm's efficiency is measured on the bases of two factors, time taken and memory used.

This is where time and space complexity comes in. Consider two algorithms solving the same problem and giving correct result. Now to determine which one is better, we will see which one takes less time and uses less memory while doing so.

NOTATIONS :-

Since computers work in fractions of a second, we cannot really measure the time efficiency of an algorithm using conventional SI units of time, so we need a different way to represent the time taken by an algorithm.

For this, we use three types of notations which show how many steps would an algorithm perform to find the result.

These three Notations are :-

1) Big (O)

2) Big(Θ)

3) Big(Ω)

Big (O):-

Big (O) notation shows the worst case scenario meaning the most time or space an algorithm would take depending on n size of input. Consider an array with n size and n values in it. Now if the array is not sorted, we would use linear search algorithm to search for a value in array. Now linear search compares target value with value at each index, now considering that our target is located at the last index (Basically) worst case. Then linear search would perform n number of comparisons to reach it, thus n steps. So using Big (O) notation, we represent this time complexity as:-

$$O(n)$$

Where n is the size of input. Which here is the size of array. Big (O) is the most commonly used notation and we would also be focusing primarily on Big (O) but let's go through others.

Big (Θ):-

Big (Θ) notation shows the average case scenario meaning the point at the mid of worst case and best case. This notation is used in cases where worst and best case are converged to a common point, however it is not used commonly so let's not go into depths of this.

Big (Ω):-

Big (Ω) notation shows the best case scenario meaning the least time or space an algorithm would take to find the result. Again it is not common standard so let's not go deep into this as well, now will go back to Big (O) which is the commonly used standard.

So now you know what $O(n)$ means in terms of time complexity, now let's see how we can use it for space complexity to judge the memory consumption.

Consider an algorithm that finds the maximum element (largest value) in an array. Again the array is of size n and is still unsorted, so we will use linear search to walk through the whole array again. Hence time complexity is again $O(n)$ but what is space complexity? How would memory usage change with respect to input size n . Well consider the following code:-

Code :-

```
int largest(const int arr[], int s){  
    int max = arr[0];  
    for (int i = 1; i < s; i++)  
        if (arr[i] > max)  
            max = arr[i];  
    return max;  
}
```

So this algorithm declares an int variable of 4 bytes called max. Now if array size or in other words " n " changes from let's say 10 to 100, the function

Date: / /

Day

would still declare a single int max to track the largest. Hence, this algorithm took the same amount of memory for size 10 and size 100. This means that this algorithm's space complexity is independent of the input size or you can say it's constant, we can represent constant complexity using Big (O) like :-

$O(1)$

As there is no "n" in the notation, it means the space complexity is not dependant on size of input.

Now let's consider a function that takes a vector and returns a reverse version of it -

Code -

```
vector<int> reverse(const vector<int> a){  
    vector<int> rev = a;  
    for(int i=0, j=rev.size()-1; i<j; i++, j--)  
        swap(rev[i], rev[j]);  
    return rev;  
}
```

Now this function returns the

reverse of a vector without modifying the original vector. Now let's find its space & time complexity. Let's start with space:-

Explanation -

We know that for a vector "a" of size "n", our function will make another vector "rev" of size "n". Therefore "n" space is used for "n" size input. So the space complexity is $O(n)$. This means that space consumption will increase as the size of input increases.

Now let's look at its time complexity :-

Explanation -

Now looking at it over the top, we can see that i and j are converging towards the middle so our brain says directly that time complexity is $n/2$, which is correct but, in Big(O) Notation the constants are

Date: 1/1

Day

ignored. So basically for any constant " C ", we can say that:-

$$O(Cn), O(n/C), O(n+C), O(n-C)$$

are all $O(n)$. Hence our vector reversal function's time complexity is $O(n)$.

COMMON COMPLEXITY LEVELS:-

You have learnt about the $O(1)$ and $O(n)$ by now. However, the complexity chart goes beyond this. Here are some commonly used complexity levels:-

- | | |
|-------------|------------------|
| 1) $O(1)$ | 2) $O(\log n)$ |
| 3) $O(n)$ | 4) $O(n \log n)$ |
| 5) $O(n^2)$ | 6) $O(n^3)$ |
| 7) $O(2^n)$ | 8) $O(n!)$ |

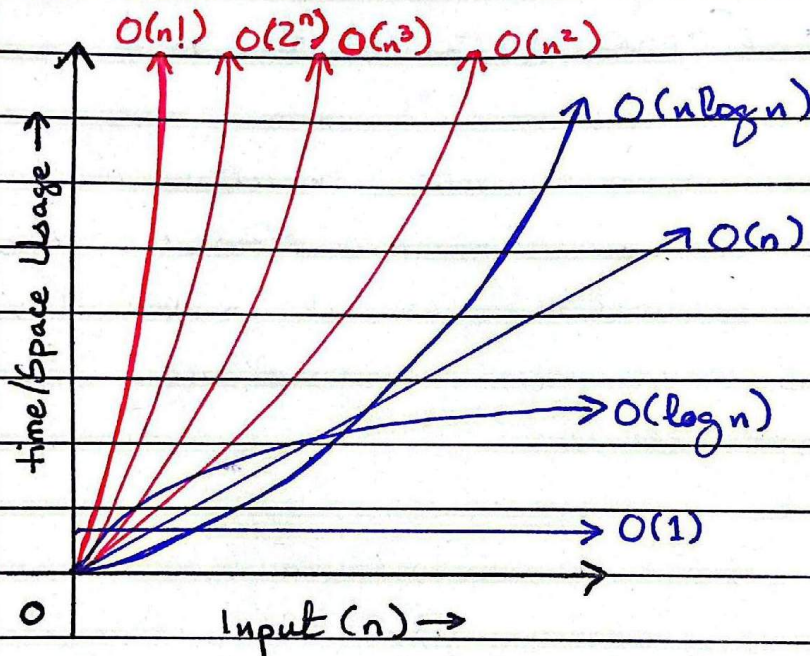
A good complexity level is considered to give a smaller value. Hence in terms of speed, the $O(1)$ is fastest and $O(n!)$ is slowest. Other relations are also compared using the same criteria:-

Look at the following:-

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

As said before, a complexity level with a smaller value is considered better.

GRAPH:-



Now let's discuss each level briefly:-

$O(1)$:-

This is constant complexity and it means that time and space usage is independent of the input size. A good example is push-back in

vectors. Its time complexity is $O(1)$.

$O(\log n)$:-

This is logarithmic complexity, it means that time and space usage increases logarithmically with respect to input size. It's faster or lighter than linear $O(n)$ and is considered one of the most efficient levels. A good example would be the binary search which we used in chapter 1. Its time complexity is $O(\log n)$, now you know why it is also called "logarithmic search".

$O(n)$:-

This is linear complexity, it means that space and time usage increases directly with respect to input size. It's considered a good level of complexity after $O(\log n)$. A good example of this is time complexity of the linear search and also time complexity of finding largest element in array.

$O(n \log n)$:-

This complexity level is faster than $O(n^2)$ but slower than $O(n)$. It is also considered a good level being the sweet spot between $O(n^2)$ and $O(n)$. A good example of this would be the time complexity of fast sorting algorithms like "merge sort", we will study this later on in chapter of sorting.

$O(n^2)$:-

This complexity lies between $O(n^3)$ and $O(n \log n)$, it is considered an averagely efficient complexity which is not too fast or too slow but as the input gets larger, it does fall into one of the slow/weak categories. Some good examples are time complexity of a ~~common~~ common sorting algorithm like bubble sort or selection sort. Other than that, Time complexity of any nested loop (consisting of two loops) is $O(n^2)$.

$O(n^3)$:-

This complexity level gives a slower/heavier result than $O(n^2)$ and is also considered an extremely inefficient one. Good example of this is time complexity of nested loops which have 3 loop (each inside the previous one).

$O(2^n)$:-

This is exponential complexity, it is one of the slowest/heaviest levels and can cause a billion cpu operations even for small inputs. A good example would be the time complexity of finding n^{th} fibonacci term using recursion.

$O(n!)$:-

This is factorial complexity, it is the worst complexity level and shall be avoided as much as possible. A good example of this is the time complexity of finding all permutations of an array/vector.

This is gonna be all for the basics of time and space complexity of an algorithm.

Here are some practice questions and tasks to strengthen your concepts -

PRACTICE EXERCISE:-

- 1) What is the time complexity of a linked list traversal?
- 2) What is the time complexity of pop-back in vectors and in linked list (Discuss how presence/absence of tail pointer in a doubly linked list affects time complexity of popback).
- 3) If a function first performs a linear search and after that a binary search, what is overall time complexity of this function?

This is it for this chapter, next up, we will look into a data structure called "stack". Till then, Good Luck!